



- Проект успешно реализован

# Finlytics

## Офлайн-менеджер финансов

Десктопное приложение для управления личными финансами с полной автономностью и конфиденциальностью данных

Платформа  
Kotlin/JVM

Архитектура  
MVVM

База данных  
SQLite

Команда разработчиков



Враженко Д.О.  
Team Lead



Хохряков А.Ю.  
Аналитик



Павлов Н.С.  
UI/UX Designer



# Состав команды и роли

Распределение ответственности и обоснование выбора ролей



**Враженко Д.О.**

Team Lead

- ✓ Постановка задач и управление командой и проектом
- ✓ Разработка общей архитектуры MVVM
- ✓ Реализация ключевых модулей системы
- ✓ Составление проектной документации

**Обоснование:** Опыт управления проектами и глубокое понимание архитектурных паттернов



**Хохряков А.Ю.**

Аналитик / Backend

- ✓ Сбор и анализ требований к системе
- ✓ Проектирование модели данных и БД
- ✓ Реализация слоя работы с данными
- ✓ Разработка бизнес-логики приложения

**Обоснование:** Сильные навыки в анализе данных и проектировании баз данных



**Павлов Н.С.**

UI/UX Designer

- ✓ Проектирование пользовательского интерфейса/опыта
- ✓ Создание макетов в Figma
- ✓ Разработка UI на Compose for Desktop
- ✓ Визуализация данных (диаграммы)

**Обоснование:** Опыт в дизайне интерфейсов и визуализации данных



**Принцип распределения:** Роли распределены на основе сильных сторон каждого участника, что обеспечило эффективное выполнение всех задач проекта в рамках методологии Kanban

# Описание проекта

Назначение, целевая аудитория и проблематика



## Что такое Finlytics?

**Finlytics** — это десктопное приложение для управления личными финансами, разработанное на языке Kotlin с использованием современных технологий.

- ✓ Учет доходов и расходов с категоризацией
- ✓ Визуализация финансовой статистики
- ✓ Фильтрация данных по временным периодам
- ✓ Полная офлайн-работа без интернета



## Проблематика

Современная жизнь требует грамотного управления личными финансами, особенно от студентов с ограниченным бюджетом.

- ✗ Существующие решения требуют интернет
- ✗ Избыточная сложность интерфейса
- ✗ Отсутствие приватности данных



## Целевая аудитория

1

### Основной сегмент

Студенты

Молодые люди, впервые сталкивающиеся с самостоятельным управлением финансами. Ищут простой инструмент для отслеживания трат.

2

### Вторичный сегмент

Профессионалы

Занятые работающие люди, ценящие приватность и скорость. Предпочитают держать данные под полным контролем.



## Ключевое преимущество

**Полная автономность и конфиденциальность** — все данные хранятся локально на устройстве пользователя без необходимости подключения к интернету.

100%

Офлайн

0

Серверов

# | Цели и требования

Функциональные и нефункциональные требования к системе



## Цели проекта

1

Разработать архитектуру данных и UI для удобного внесения и редактирования финансовых операций

2

Реализовать механизм категоризации доходов и расходов с возможностью добавления пользовательских категорий

3

Внедрить систему фильтрации и анализа данных с визуализацией результатов в виде диаграмм и отчетов



## Функциональные требования



CRUD операций



Управление категориями



Фильтрация по периодам



Круговая диаграмма



Финансовая сводка



История операций



## Нефункциональные требования

### Кроссплатформенность



Windows 10+, macOS 11+, Linux с JRE

### Производительность



Отклик <500мс, запуск <3 сек

### Надежность



Корректное завершение, целостность данных

### Безопасность



Защита от SQL-инъекций, локальное хранение



## Результаты тестирования

14/14

Требований выполнено

10/10

Тестов пройдено

# Архитектура программного продукта

Многослойная архитектура MVVM для десктопного приложения



## Обоснование выбора MVVM

- ✓ **Соответствие типу приложения**  
Идеально для монолитных десктопных приложений без излишнего усложнения
- ✓ **Разделение ответственности**  
Четкое разделение UI, логики и данных делает код понятным и расширяемым
- ✓ **Гибкость и расширяемость**  
Возможность изменять слои независимо друг от друга
- ✓ **Реактивность**  
Kotlin Flows обеспечивают автоматическое обновление UI



## Преимущества архитектуры



Тестируемость



Поддерживаемость



Масштабируемость



Реактивность

## Структура архитектуры



### UI Layer (View)

Compose for Desktop

Отображение

Декларативный UI, реактивное обновление через StateFlow, кастомные компоненты



### ViewModel Layer

Kotlin Coroutines & Flow

Состояние

Управление состоянием приложения, бизнес-логика представления, реактивные потоки данных



### Repository Layer

FinanceRepository

Данные

Абстракция доступа к данным, единая точка входа, координация операций



### Data Layer

SQLite + JDBC

Хранение

Локальное хранение данных, SQL-запросы с PreparedStatement, транзакции

# Наиболее значимые риски

Оценка рисков, матрица и планы реагирования

## Матрица рисков

| Риск                  | Вер-ть | Посл-я | Важ-ть | Зона    |
|-----------------------|--------|--------|--------|---------|
| Сложность UI          | 8      | 6      | 48     | Красная |
| Производительность БД | 6      | 8      | 48     | Красная |
| Изменения требований  | 6      | 8      | 48     | Красная |
| Отсутствие JRE        | 5      | 9      | 45     | Красная |
| Повреждение БД        | 3      | 10     | 30     | Желтая  |



## Реализованные меры

- ✓ Внутренний воркшоп по Compose для Desktop
- ✓ Индексы в БД для полей date
- ✓ Демонстрация инкрементов заказчику
- ✓ Резервное копирование БД (.bak)

## Планы реагирования на топ-риски

### 1 Сложность UI на Compose

**Стратегия:** Снижение

Основной план: Внутренний воркшоп, запас времени на сложные компоненты

Отходной план: Упрощенный макет диаграммы со стандартными компонентами

### 2 Производительность БД

**Стратегия:** Снижение

Основной план: Индексы для полей date, EXPLAIN QUERY PLAN

Отходной план: Кэширование результатов фильтрации в ViewModel

### 3 Изменения требований

**Стратегия:** Принятие

Основной план: Демонстрация инкрементов, согласование изменений

Отходной план: Задача «Доработки» с резервом 1-2 дня

### 4 Отсутствие JRE

**Стратегия:** Уклонение

Основной план: Нативные установщики с встроенной JRE

Отходной план: Явное указание требований в README

# | Стек технологий

Используемые технологии и обоснование выбора



## Kotlin

Язык программирования

Современный, лаконичный и безопасный язык для платформы JVM. Идеально подходит для разработки десктопных приложений.

Null-safety

Coroutines

DSL



## SQLite

База данных

Легковесная, встраиваемая реляционная база данных. Не требует отдельного сервера, идеальна для офлайн-приложений.

Встраиваемая

Нулевая конфигурация

Транзакции



## Compose for Desktop

UI-фреймворк

Современный декларативный фреймворк для создания нативных UI. Меньше кода, интуитивная разработка, интеграция с Flow.

Декларативный

Реактивный

Кроссплатформенный



## Gradle + Kotlin DSL

Сборка и зависимости

Стандарт де-факто для сборки Kotlin-проектов. Kotlin DSL обеспечивает типовую безопасность и автодополнение в IDE.

Типовая безопасность

Автодополнение



## Coroutines & Flow

Управление состоянием

Реактивные потоки данных для автоматического обновления UI. Асинхронные операции с БД без блокировки главного потока.

StateFlow

Асинхронность



## Дополнительные инструменты



Figma (дизайн)



Git + GitHub



Miro (Kanban)



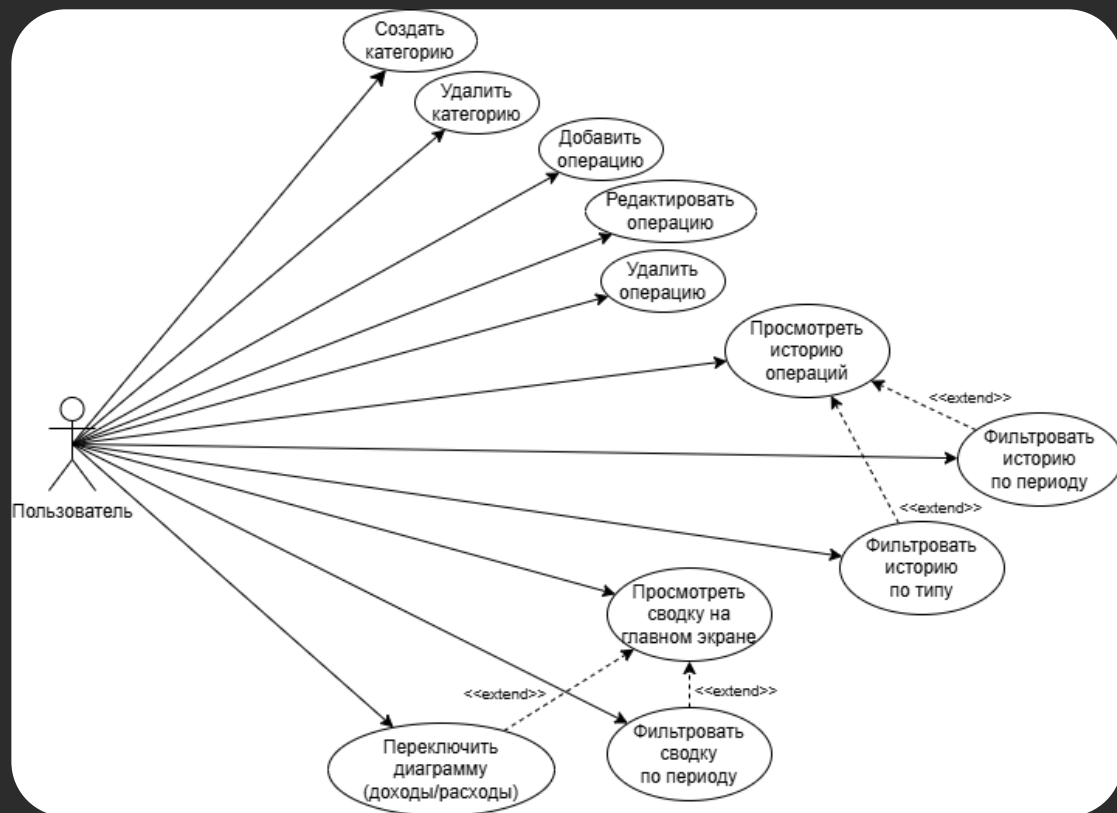
IntelliJ IDEA

# Диаграммы процессов

Use Case, DFD и BPMN диаграммы проекта



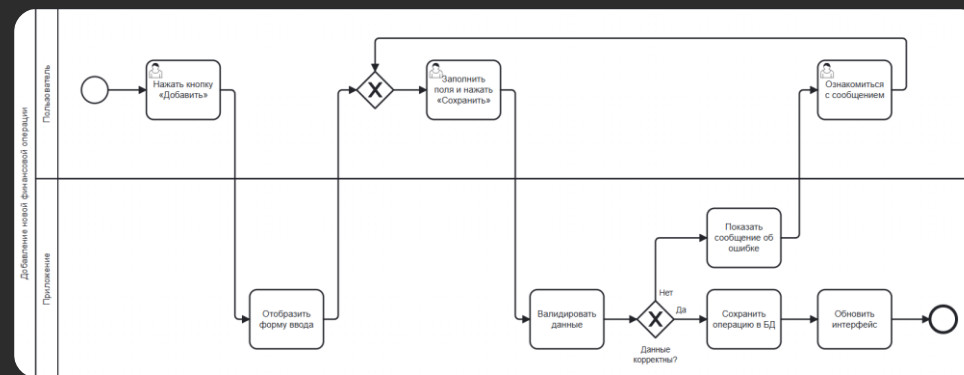
## Диаграмма вариантов использования



## Диаграмма DFD



## BPMN: Добавление операции





# Описание функционала

Основные возможности приложения Finlytics



## Управление операциями



### Добавление

Создание операций дохода/расхода с суммой, категорией, датой и описанием



### Редактирование

Изменение любых параметров существующих операций



### Удаление

Удаление операций с подтверждением и пересчетом статистики



Валидация всех полей перед сохранением



## Управление категориями

- ✓ Предустановленные категории при первом запуске
- ✓ Создание пользовательских категорий
- ✓ Удаление неиспользуемых категорий
- ✓ Защита от удаления используемых категорий



## Фильтрация и аналитика

### Временные периоды

День

Неделя

Месяц

Год

Все время

### Типы операций

Все

Доходы

Расходы

### Финансовая сводка

Доходы

Общая сумма

Расходы

Общая сумма

Баланс

Разница



## Визуализация



### Круговая диаграмма

Распределение операций по категориям с процентами



### Список категорий

Детализация по каждой категории с суммами



### Гонка с призраком

Сравнение расходов с предыдущим периодом



## Интерфейс приложения



### Экран "Обзор"

Главный экран со статистикой

- ✓ Круговая диаграмма расходов/доходов
- ✓ Финансовая сводка (баланс, доходы, расходы)
- ✓ Фильтрация по периодам
- ✓ Прогресс-бар "Гонка с призраком"



### Экран "История"

Полный список операций

- ✓ Список всех операций с датами
- ✓ Кнопки редактирования и удаления
- ✓ Фильтрация по типу и периоду



### Экран "Настройки"

Управление категориями

- ✓ Список категорий доходов и расходов
- ✓ Добавление новых категорий
- ✓ Удаление категорий с проверкой

# Тестирование

Методы, сценарии и результаты тестирования

## Методы тестирования

### Модульное

JUnit 5 для тестирования отдельных компонентов

### Интеграционное

Проверка взаимодействия модулей

### UI-тестирование

Compose UI Test для проверки интерфейса

### Нагрузочное

Apache JMeter для проверки производительности

## Результаты тестирования

10/10

Тест-кейсов пройдено

100%

Успешность

- ✓ Все функциональные требования покрыты тестами
- ✓ Время отклика <300мс (требование <500мс)
- ✓ Валидация данных работает корректно
- ✓ Защита от SQL-инъекций подтверждена

Окружение: Windows 11 Pro, JRE 21, разрешение 1920x1080

## Тестовые сценарии (Test Cases)

### ТС-01: Добавление операции

Позитивный сценарий создания расхода

Пройден

### ТС-02: Валидация суммы

Негативный сценарий с отрицательной суммой

Пройден

### ТС-03: Редактирование операции

Изменение существующей записи

Пройден

### ТС-04: Удаление операции

Удаление с подтверждением

Пройден

### ТС-05: Создание категории

Добавление пользовательской категории

Пройден

### ТС-06: Защита категорий

Блокировка удаления используемой категории

Пройден

### ТС-07: Фильтрация по периоду

Аналитика за выбранный промежуток

Пройден

### ТС-08: Переключение диаграммы

Переключение между доходами и расходами

Пройден

### ТС-09: Валидация даты

Ограничение ввода дат в будущее

Пройден

### ТС-10: Прогресс-бар "Призрак"

Сравнение с предыдущим периодом

Пройден

# Развертывание

Процесс установки и требования к системе



## Формат распространения



### Finlytics.jar

Исполняемый JAR-файл

- ✓ Содержит все зависимости
- ✓ Автоматическая инициализация БД
- ✓ Портативность (не требует установки)

💡 Альтернатива: Нативные установщики (.msi, .dmg, .deb) с встроенной JRE через Gradle jpackage



## Требования к системе



### Windows

Windows 10 и выше



### Linux

Дистрибутивы с JRE



### macOS

macOS 11 и выше



### Java Runtime Environment

Версия 8 или выше

✓ Кроссплатформенность проверена: Windows 11, Ubuntu 22.04



## Процесс установки

1

### Скачивание

Загрузите Finlytics.jar из GitHub

[github.com/Hohrandrey/Finlytics](https://github.com/Hohrandrey/Finlytics)

2

### Проверка JRE

Убедитесь, что установлена Java 8+

`java -version`

3

### Запуск

Варианты запуска приложения

`java -jar Finlytics.jar`

Двойной клик по файлу

4

### Инициализация

При первом запуске автоматически создается база данных со стандартными категориями

# Документация разработчика

KDoc комментарии и генерация документации



## KDoc документирование

Все публичные классы, функции и свойства имеют подробные KDoc-комментарии с описанием назначения, параметров, возвращаемых значений и возможных исключений.

```
/**
 * ViewModel для управления состоянием
 * приложения и бизнес-логикой
 *
 * @param repo Репозиторий для доступа к данным
 */
class FinanceViewModel(
    private val repo: FinanceRepository
) { ... }
```



## Инструмент Dokka

Dokka — официальный инструмент генерации документации для Kotlin. Создает HTML-документацию из KDoc-комментариев.

- ✓ Автоматическая генерация из Gradle
- ✓ HTML-формат с навигацией
- ✓ Интеграция с IntelliJ IDEA

```
./gradlew dokkaHtml
```



## Структура документации

The screenshot shows the documentation for the `FinanceState` class in the `Finlytics 1.0` project. The left sidebar lists the project structure, including `Finlytics`, `database_request`, `main`, `models`, `repository`, `ui`, `ui.components`, `ui.screens`, `ui.theme`, `ui.theme.icons`, `ui.theme.icons.finityticsiconpack`, `utils`, and `viewmodel`. The main content area displays the `FinanceState` class with its data members and a constructor. The members include `operations`, `totalIncome`, `totalExpenses`, `balance`, `expensesByCategory`, `incomeCategories`, `expenseCategories`, and `editingCategory`. The constructor takes `operations` and `totalIncome` as parameters. A description below the code states: "Класс состояния приложения, содержащий все данные, необходимые для отображения UI. Состояние является иммутабельным (immutable) и обновляется через Flow в архитектуре MVVM." The "Members" and "Constructors" sections are also visible.



Соответствие стандарту: ГОСТ 34.201-2020 на документирование ПО

# Документация пользователя

Встроенная помощь и GitHub Wiki



GitHub Wiki



github.com/Hohrandrey/Finlytics  
wiki

## Структура Wiki



Home

Обзор приложения и разделов документации



Безопасность

Описание защиты данных и аутентификации



Интерфейс

Детальное руководство по страницам и элементам интерфейса



Управление данными

Правила работы с транзакциями и категориями



Установка и запуск

Системные требования и инструкции по установке



## Встроенная помощь в UI



### ToolTips (всплывающие подсказки)

Все ключевые элементы управления снабжены подсказками, поясняющими их назначение

Кнопки

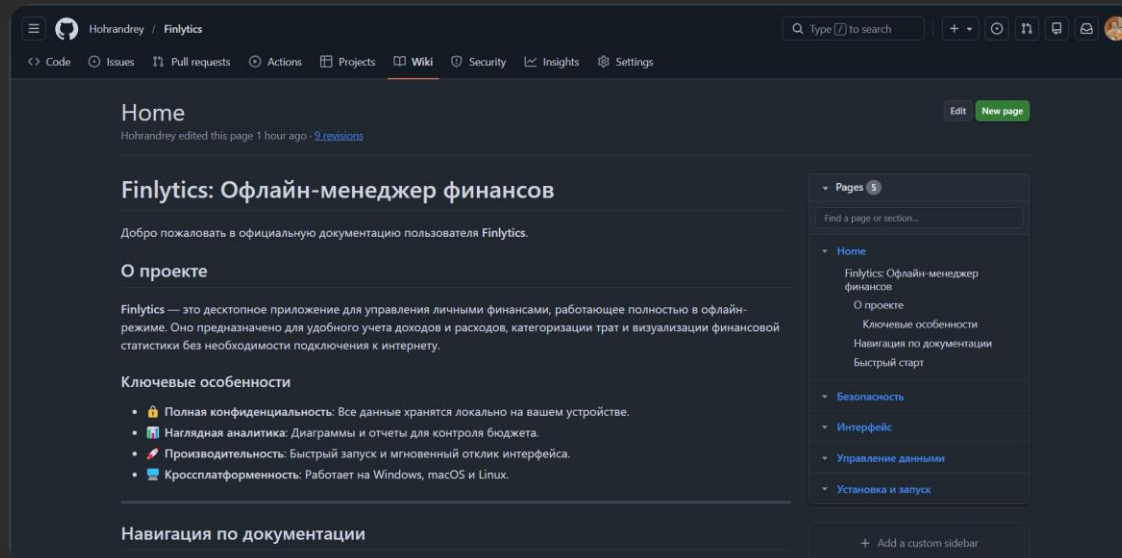
Поля ввода

Фильтры



### Сообщения об ошибках

Понятные сообщения при некорректном вводе данных (сумма, дата, категория)



# Достоинства и недостатки

Сравнительный анализ с существующими аналогами

## Сравнительный анализ

| Параметр             | Finlytics   | Аналоги   |
|----------------------|-------------|-----------|
| Офлайн-работа        | Полная      | Частично  |
| Приватность          | Локальная   | Облачная  |
| Сложность UI         | Простой     | Средний   |
| Кроссплатформенность | Win/Mac/Lin | Мобильные |
| Стоимость            | Бесплатно   | Freemium  |

## Достоинства

-  **Полная автономность**  
Работа без интернета, данные только на устройстве
-  **Конфиденциальность**  
Локальное хранение, никаких облачных сервисов
-  **Простота использования**  
Интуитивный интерфейс без лишних функций
-  **Кроссплатформенность**  
Работа на Windows, macOS и Linux
-  **Открытый исходный код**  
Возможность изучения и модификации

## Недостатки

-  **Требуется JRE**  
Необходимость установки Java Runtime Environment
-  **Нет синхронизации**  
Отсутствие синхронизации между устройствами
-  **Ограниченная аналитика**  
Базовые отчеты без прогнозирования
-  **Только десктоп**  
Отсутствие мобильной версии приложения

## Области для улучшения

- Создание нативных установщиков с встроенной JRE
- Добавление экспорта/импорта данных
- Расширенная аналитика с трендами
- Разработка мобильной версии

✓ Проект успешно завершен

# Спасибо за внимание!

Finlytics — готовое решение для управления личными финансами  
с полной автономностью и конфиденциальностью



**Открытый код**  
GitHub репозиторий



**Приватность**  
Локальное хранение



**Готов к использованию**  
JAR-файл для запуска



**github.com/Hohrandrey/Finlytics**

Исходный код, документация и релизы

**В**

Враженко Д.О.

**Х**

Хохряков А.Ю.

**П**

Павлов Н.С.